

LAB 4: Shell Scripting Evaluation 2B

Jibon Naher¹ and Samnun Azfar²

¹Lecturer,CSE

²Junior Lecturer,CSE

December 11, 2025

General Instructions

1. Create a separate file for each task (e.g., ID_task1.sh, ID_task2.sh etc.).
2. Use the PC from the lab.
3. Submit your mobile at the front.

1 Task 1: Calculate the Sum and Average of Array Elements

Concepts Practiced: Arrays, loops, arithmetic expressions. **Goal:** Write a shell script that calculates the total sum and average of given integer elements.

1.1 Requirements

*

1. Read n integer elements for the array from the user.
2. Initialize `sum=0`.
3. Use a loop to accumulate the sum of all elements.
4. Compute the average as `sum/n`.
5. Display both the sum and average.

1.2 Expected Output (Example)

*

```
1 Enter the number of elements: 4
2 Enter element 1: 10
3 Enter element 2: 20
4 Enter element 3: 30
5 Enter element 4: 40
6 Sum of all elements: 100
7 Average of elements: 25
```

Task 2:

Objective

In this labtask, you will write a Bash script to automate the build process of a C++ project titled "Vid2Console".

While the project is written in C++, **you do not need to know how to write C++ code**. Your task is to create a script that installs necessary dependencies, manages build directories, handles compilation flags (Debug/Release), and links the final executable. To start with, clone the following **Git Repo** :

```
1 git clone -b lab4 https://github.com/azfarus/vid2console.git
```

1. Project Structure

Assume you are in the root directory of the project. The file structure is as follows:

```
project_root/
  include/          # Header files (.h)
  lib/              # Library implementation files (.cpp)
    printer.cpp
    rendererer.cpp
    vid2arr.cpp
    framegenerator.cpp
  src/
    vid2console/
      main.cpp      # The entry point
  build.sh          # YOU are creating this file
```

2. Compilation Basics (Read Carefully)

Since we are manually invoking the compiler (g++), you need to understand the pipeline. C++ compilation happens in two distinct stages:

Stage A: Compiling to Objects (.o)

We take a source file (.cpp) and turn it into a binary object file (.o).

- **The Command:** `g++ -c source.cpp -o output.o`
- **Flag -c:** Tells g++ to "compile only, do not link yet."
- **Flag -I <path>:** The compiler needs to know where the header files (.h) live. Since our headers are in the `include` folder, we must pass `-I include`.
- **External Libs (OpenCV):** This project uses OpenCV. Instead of guessing where OpenCV is installed, we use a tool called `pkg-config`.
 - To get compilation flags: `pkg-config --cflags opencv4`

Stage B: Linking (.exe or binary)

We take *all* the .o files created in Stage A and glue them together to create the final executable.

- **The Command:** `g++ file1.o file2.o ... -o final_executable`
- **Linking Libraries:** We need to tell the linker to include the actual OpenCV library code.
 - To get linker flags: `pkg-config --libs opencv4`

3. Task Requirements & Steps

Create a file named `build.sh` and implement the following:

Step 1: Safety and Dependencies

You must start the script by ensuring it exits on errors and installing the necessary tools.

Required Implementation for Step 1: Copy the following code exactly to start your script. These lines will install the required dependencies for compilation. :

```
1  #!/usr/bin/bash
2
3  set -e #sets the script to exit when any error occurs
4
5  # Update package lists
6  sudo apt update
7
8  # Install ffmpeg and related libraries
9  sudo apt install -y ffmpeg libavcodec-dev libavformat-dev libavutil-dev
10 libswscale-dev
11
12 # Install OpenCV and development headers
13 sudo apt install -y libopencv-dev
14
15 # Install Build Essentials (GCC, G++, Make, etc.)
16 sudo apt install -y build-essential
```

Step 2: Variable Definitions

Define variables at the top of your script (after the installations) to keep it clean.

1. **Directories:** Define variables for an Object directory (`build/obj`) and a Binary directory (`build/bin`).
2. **OpenCV Flags:**
 - Create a variable `OPENCV_COMPILE_FLAGS` that stores the result of `$(pkg-config --libs opencv4 --cflags opencv4)`.
 - Create a variable `OPENCV_LINKING_FLAGS` that stores `$(pkg-config --libs opencv4)`.
3. **Standard Flags:** Initialize a variable `CXX_FLAGS` as an empty string.

Step 3: Argument Parsing (Loops and Conditionals)

Your script should accept command-line arguments (e.g., `./build.sh clean debug`). Iterate through all arguments passed to the script (run a for loop through "\$@") and implement the following logic:

1. **Clean:** If the argument is `clean`:
 - Check if the `build` directory exists. Use the syntax `if [ -d build ]`
 - If it does, remove it.
2. **Debug:** If the argument is `debug`:
 - Set `CXX_FLAGS` to `-g -O0` (Includes symbols, no optimization).
3. **Release:** If the argument is `release`:
 - Set `CXX_FLAGS` to `-s -O3` (Strip symbols, maximum optimization).

Step 4: Directory Preparation

After checking arguments, ensure your build directories (`build/obj` and `build/bin`) exist.

Hint: Use `mkdir` with the flag that creates parents if needed and doesn't complain if they already exist.

Step 5: The Compilation Phase

You need to compile 5 specific files. Print a message to the console saying "Compiling...".

General Syntax for this step:

```
1 g++ -c $CXX_FLAGS <source_file> -o <output_object> -I <include_dir> \  
2 $OPENCV_COMPILE_FLAGS -std=c++17  
3
```

Your Task: Write the specific commands to compile the following mappings:

Source File (.cpp)	Output Object (.o) inside build/obj
src/vid2console/main.cpp	main.o
lib/printer.cpp	printer.o
lib/renderer.cpp	renderer.o
lib/vid2arr.cpp	vid2arr.o
lib/framegenerator.cpp	framegenerator.o

Step 6: The Linking Phase

Now, link all those object files into one executable. Print a message saying "Linking...".

Syntax:

```
1 g++ <list of all .o files> $OPENCV_LINKING_FLAGS -o <final_binary_path> -  
2 std=c++17
```

1. The final binary should be placed in `build/bin/vid2console`.
2. After linking, use `chmod` to ensure the resulting binary has execute permissions (777).

4. Testing Your Script

Run the following commands to verify your script works:

1. **Clean build:** `./build.sh clean` (Should remove build folder)
2. **Debug build:** `./build.sh debug` (Should build without errors)
3. **Run it:** `./build/bin/vid2console`